QIBIJI Small RTU V1.1 电机电流测定电路与固件分析

从 50 mΩ 分流电阻、LM358 放大、ADC/DMA 到堵转停机与上报

硬件与嵌入式代码联合分析

2026-10-24

结论摘要
1. 分析范围与版本证据
1.1 硬件与代码来源
1.2 关键源码哈希
1.3 证据等级
2. 硬件电路
2.1 电机驱动与取样位置
2.2 U4A 放大器
2.3 输出 RC 与 ADC 入口
2.4 ADC 引脚与参考
2.5 理论量程与耗散
3. ADC 与 DMA 配置
3.1 初始化顺序
3.2 DMA 缓冲布局
3.3 时间预算
3.4 定时器
4. 电流算法
4.1 代码转换常数
4.2 原始块平均
4.3 冷启动零点
4.4 单次电流
4.5 25 点滑动平均
4.6 零点慢跟踪
5. 堵转与过流保护
5.1 判定路径
5.2 时间估算
5.3 停机和保护属性
6. 固件结构与代码位置
7. 显示、属性与事件的数据一致性
7.1 LCD
7.2 4G 属性
7.3 堵转事件

8. 阈值配置与 EEPROM
8.1 默认值与存储位置
8.2 两条配置入口
8.3 重要缺陷：小于 1000 mA 的重启迁移
9. 代码审查发现
P0：安全或功能正确性
P1：测量可靠性与诊断
P2：可维护性与代码质量
10. 标定与测试方案
10.1 硬件静态检查
10.2 零点与噪声
10.3 多点电流标定
10.4 阶跃和堵转测试
10.5 阈值持久化回归
10.6 故障注入
11. 建议的代码级改进
11.1 单位与配置
11.2 真实时间窗口
11.3 状态接口
11.4 诊断事件
11.5 编译期自检
12. 关键结论
附录 A　关键源码摘录
A.1 ADC 换算常数
A.2 零点与单次样本
A.3 25 点 FIFO
A.4 堵转分支
A.5 停机动作
附录 B　原始原理图局部与整页
附录 C　报告资产

结论摘要

这套电机电流测量采用**低边 50 mΩ 分流电阻 + LM358 非反相放大 + STM32F103 的 ADC1_CH10**。硬件总增益为 2.0，满量程理论值约为 33 A，代码中的 `ADC2CURT ≈ 8.057 mA/LSB` 与当前电阻网络相符。

固件使用“连续 ADC + DMA 循环缓冲”取得数据，每 20 ms 在主循环处理一次：

- 先把最近 256 个 CH10 样本求平均，得到约 16.1 ms 的块平均；
- 电机停止时用 16 个块样本建立零点，约 320 ms 后才允许电机运行；
- 每个块样本扣除零点后换算为 `sample_ma`；
- 25 个 `sample_ma` 进入滑动 FIFO，约 500 ms 窗口用于显示与属性上报；
- 堵转保护不等待 500 ms 平均，而是使用短期 `sample_ma`，连续 10 次超阈值后请求停机；
- 停止原因以 `stall` 与“电机停止事件”两条路径进入上报队列。

当前实现有明确的默认硬件/软件值：`R43=50 mΩ`、`GAIN=2.0`、ADC 时钟 12 MHz、20 ms 处理节拍、默认阈值 3500 mA、连续 10 次判定。最主要的风险是：

1. **阈值迁移缺陷**：通过 4G 或 LCD 设置低于 1000 mA 的有效值后，重启会被 `eepromToPara()` 当成旧单位并乘以 100。
2. **保护速度有限**：软件判定约 180–200 ms，再加主循环拥塞和继电器释放时间，不能替代快速硬件限流或熔断器。
3. **ADC 过压边界**：LM358 由 +5 V 供电，而 STM32 ADC 只允许约 3.3 V；R29 可限流，但 ADC 端没有专用钳位。
4. **诊断信息不足**：零点未 ready、ADC 饱和、分流断线、运放饱和等故障没有独立状态或告警。
5. **多处注释已过期**：代码注释中的 300 ms、4 通道、128 次采样等与实际实现不一致。

> 本报告基于硬件 PDF/.NET 与当前工作区固件源码。由于没有 PCB 布局文件，无法验证分流电阻的 Kelvin 接法、模拟地回流、热铜面积、RV 输入端保护以及继电器触点噪声耦合。

1. 分析范围与版本证据

1.1 硬件与代码来源

类别	文件/版本
原理图	`hard/PCB_Project_small_rtu_V1.1.pdf`，6 页
网表	`hard/mcu(1).NET`
代码仓库 HEAD	`fbbd4090483749fee57844bde7d7c50e152d611f`
HEAD 日期	2026-10-20 16:56:56 +08:00
HEAD 说明	`修订增加急停`
关键代码	`HARDWARE/ADC/adc.c`、`adc.h`、`DC_MOTOR/dc_motor.c`、`TIMER/timer.c`

分析时，ADC、电机、定时器、寄存器、事件和上报相关文件均无未提交修改；工作区仅显示 `USER/SRTUV10.uvguix.Administrator` 被修改，以及新增的 `hard/` 报告资料目录。

1.2 关键源码哈希

文件	SHA-256
`HARDWARE/ADC/adc.c`	`5748EA9AA58F54F8205C3C03BA4F8331A65298C036D76FC071AB40CF20AE0718`
`HARDWARE/ADC/adc.h`	`8743D57E0C971C67D517CAF06C2E834D3EFDF0879F8BD0D9F60178691F570069`
`HARDWARE/DC_MOTOR/dc_motor.c`	`26B2294EBA49AF52168636F0FD8286474A3FED833FE6EB6029C9E7F9CCBE6F87`
`HARDWARE/DC_MOTOR/dc_motor.h`	`0E1BA2B90635FBCC518044571D5795CC04B20C237B718D3FB47955D661567B8E`
`HARDWARE/TIMER/timer.c`	`56D26E60114472975DEBA9B2ABC9F3F666C61DC6B2F06232BD7769B233BC4F2F`
`USER/main.c`	`72087931FCE5C38F314256C9770E4FD809805F1140B93EC86A3D62DA9A1F9074`
`HARDWARE/REG/reg.c`	`A1CCD552AF0E1671FB73CA9097B7A1068870A3337E5168DB9C91D479699C7BA9`
`HARDWARE/GPRS/event_pub.c`	`CDE1F6985796D148C487AF55426AB528A5395E10069350F3C0D9A4902CDD6273`
`HARDWARE/GPRS/property_report.c`	`00C124BA59B02DD69D962C689EF86DFD272BD2B629F91F6938FE50F46D8CDED5`

1.3 证据等级

- **直接连接**：来自 `.NET` 的元件值、引脚和网络名，可靠。
- **模拟关系**：根据电阻网络和 LM358 电路拓扑计算，可靠但未包含容差。

- **固件行为**：根据当前代码逐行推导；不包含编译器优化、未展示的启动代码或现场 PCB 细节。
- **精度结论**：只是理论值，必须通过样机多点标定后才能作为计量参数。

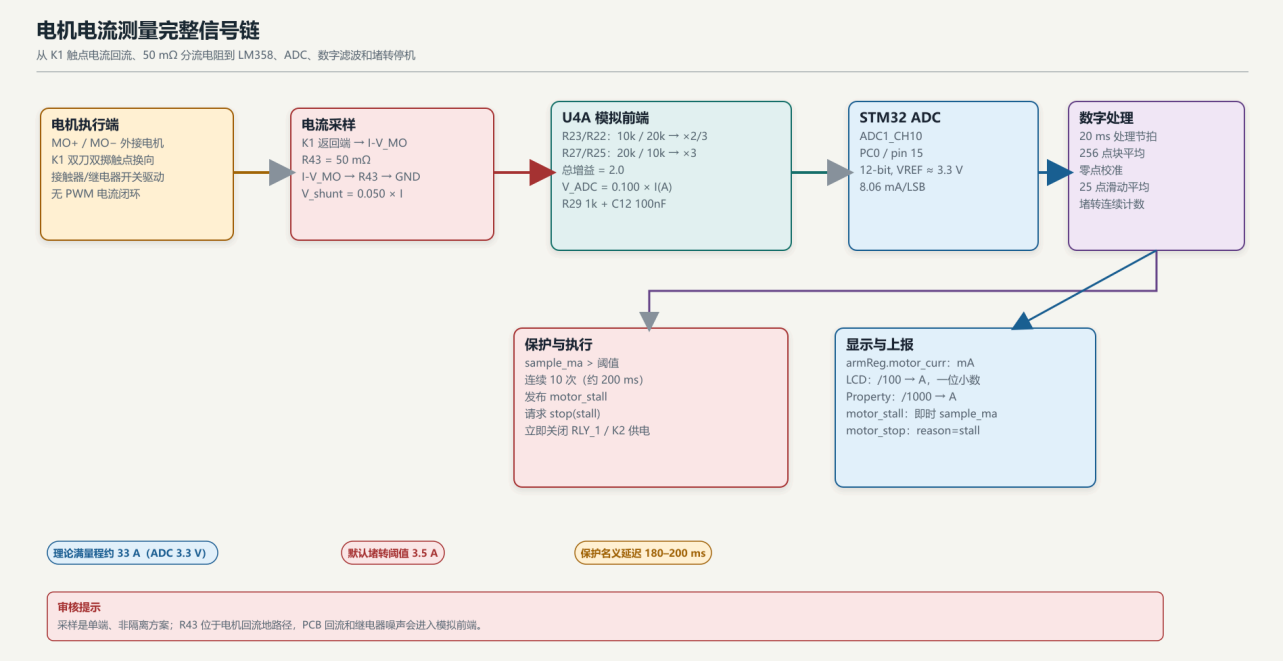


图 1　从电机回流到数字处理与堵转停机的完整信号链。

2. 硬件电路

2.1 电机驱动与取样位置

电机通过 K1 双刀双掷继电器触点换向，`MO+`、`MO−` 经 CN1 外接。固件并不直接 PWM 驱动电机，而是通过两个继电器：

- `RLY_1 = PD1`，控制 K2；
- `RLY_2 = PD3`，控制 K1 方向；
- 上升：先接通 `RLY_2`，延时 5 ms，再接通 `RLY_1`；
- 下降：先断开 `RLY_2`，延时 5 ms，再接通 `RLY_1`；
- 停止：关闭 `RLY_1`。

这种“先设方向、后给负载供电”的顺序有助于降低带载换向触点电弧。

K1 的回流端形成网络 `I-V_MO`，经 R43 50 mΩ 到 GND。分流电阻采用 R2512 大封装。`I-V_MO` 同时送入 U4A 模拟前端。

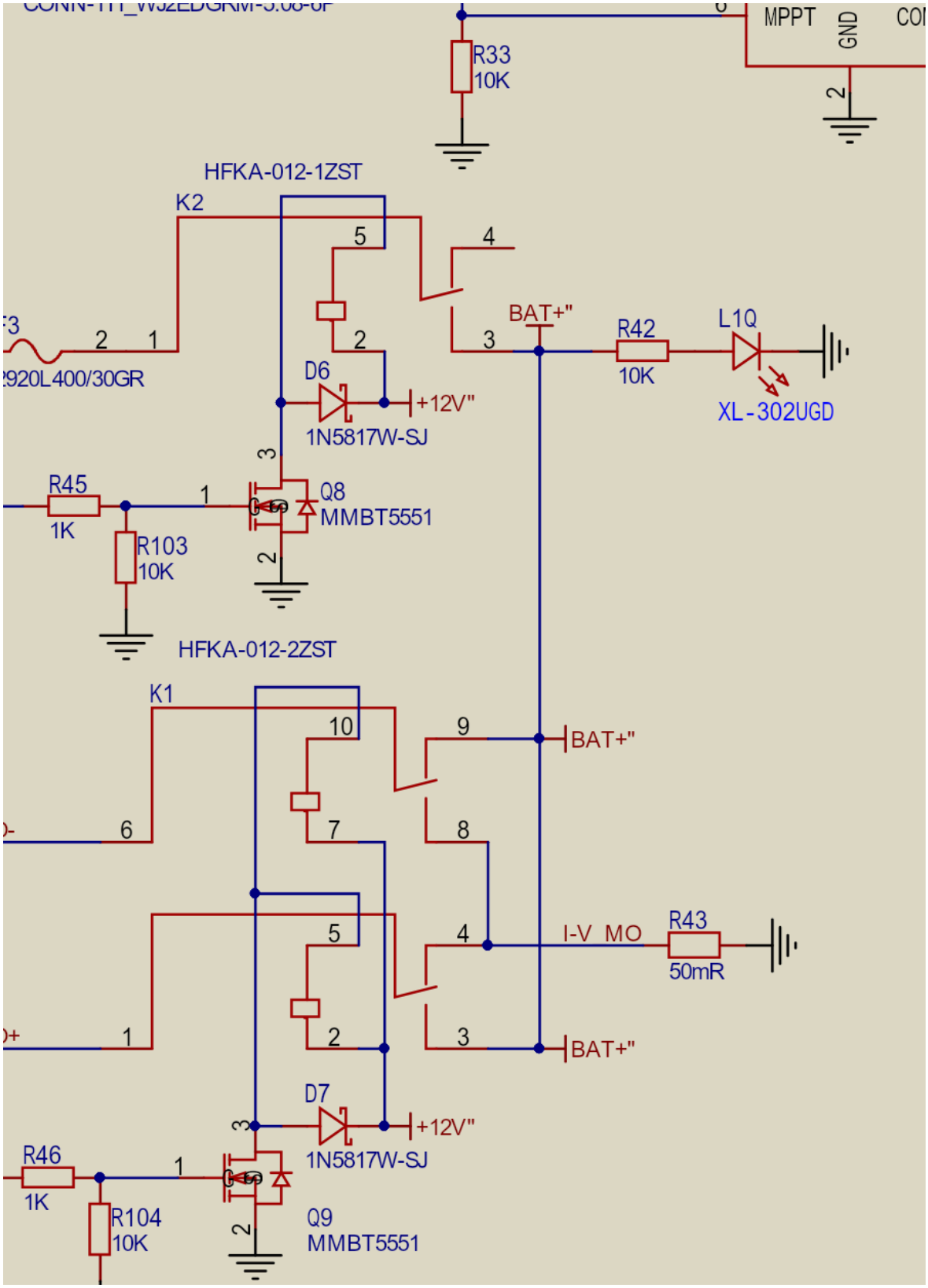


图 2　K1、MO± 与 R43 电流取样路径。

2.2 U4A 放大器

U4A 属于双运放 LM358DR2G 的 A 通道。与电流测量有关的元件：

元件	值	连接/作用
R43	50 mΩ	电机回流分流电阻

元件	值	连接/作用
R23	10 kΩ	`I-V_MO` → U4A pin 3 串联电阻
R22	20 kΩ	U4A pin 3 → GND
R25	10 kΩ	U4A pin 2（反相输入）→ GND
R27	20 kΩ	U4A pin 1（输出）→ U4A pin 2
R29	1 kΩ	U4A 输出 → ADC10 串联限流
C12	100 nF	ADC10 → GND
U4 pin 8	+5 V	运放供电
U4 pin 4	GND	运放地

该电路分为两级等效增益：

1. R23/R22 构成对 `I-V_MO` 的衰减：

$$V_+ = \frac{R22}{R23 + R22} V_{\text{shunt}} = \frac{20}{10 + 20} V_{\text{shunt}} = \frac{2}{3} V_{\text{shunt}}$$

2. R27/R25 构成同相放大器：

$$G_{\text{amp}} = 1 + \frac{R27}{R25} = 1 + \frac{20}{10} = 3$$

3. 总增益：

$$G_{\text{total}} = \frac{2}{3} \times 3 = 2$$

因此：

$$V_{\text{ADC}} = G_{\text{total}} V_{\text{shunt}} = 2 \times 0.050 I_{\text{MO}} = 0.100 I_{\text{MO}} \quad [\text{V/A}]$$

代码中的 `GAIN=2.0` 是对**总增益**的定义，不是单独指 R27/R25 的 3 倍。

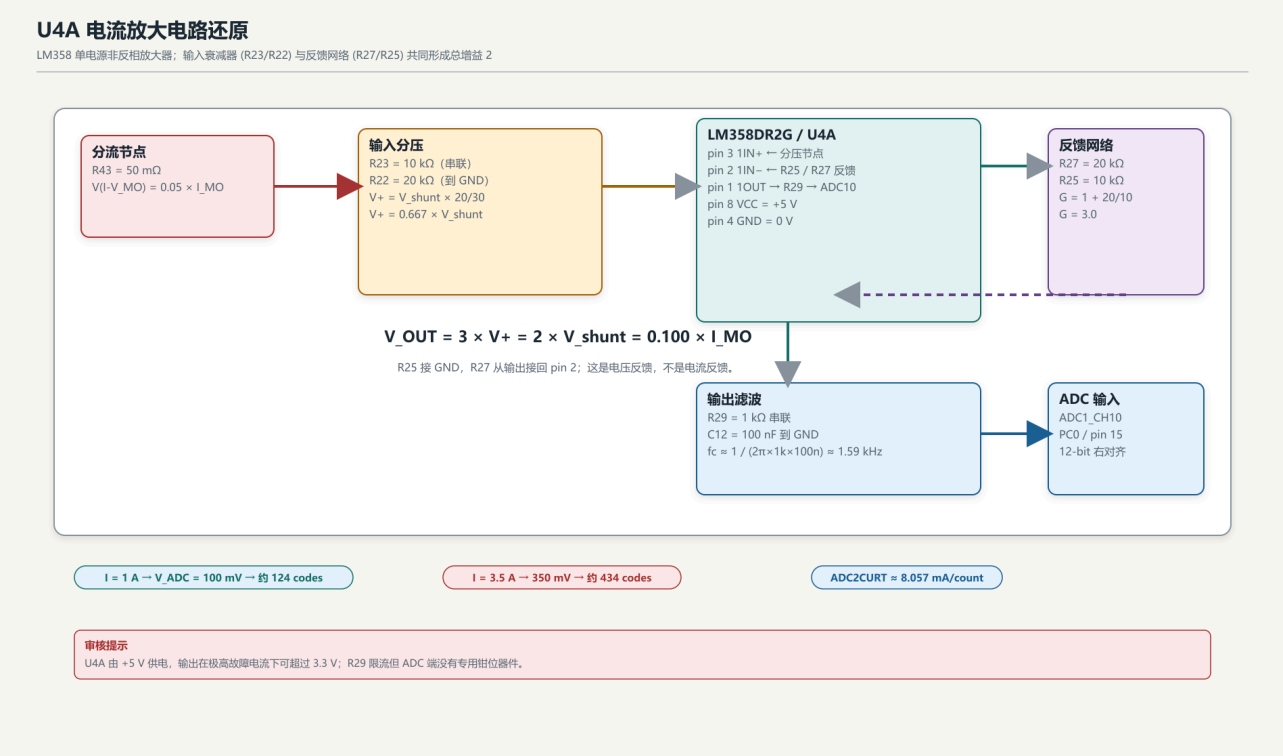


图 3　U4A 电流放大器的网表还原。

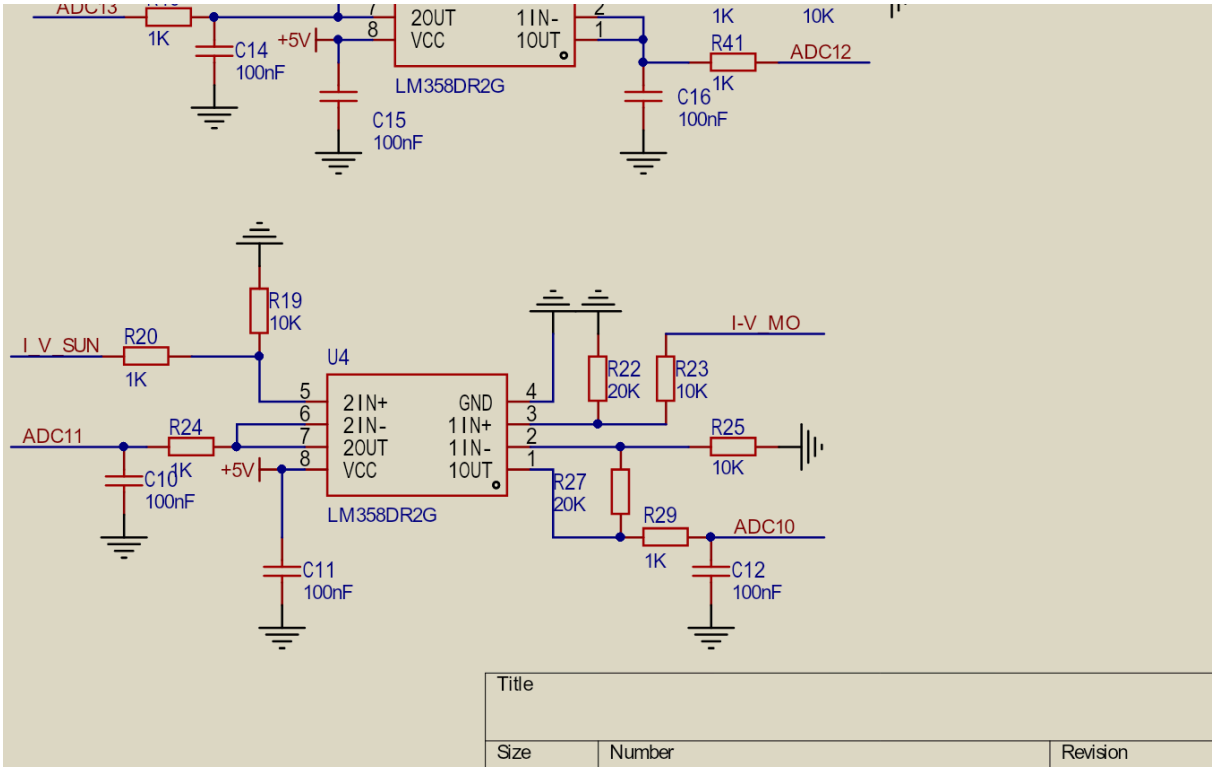


图 4　原始原理图中 LM358、分流网络和 ADC10 局部。

2.3 输出 RC 与 ADC 入口

R29 1 kΩ 与 C12 100 nF 构成单极点低通：

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \times 1000 \times 100 \times 10^{-9}} \approx 1592\ \text{Hz}$$

100 nF 的直接负载由 R29 隔离。ADC 采样时间设置为 239.5 个 ADC 周期，约 `239.5 / 12 MHz = 19.96 µs`，远长于 1 kΩ 对采样电容的短时充电时间，因此采样建立不是主要瓶颈。

但 R29/R27 并不能阻止 U4A 在故障时输出高于 3.3 V。LM358 的 +5 V 电源使其输出摆幅可能显著高于 STM32 ADC 的绝对最大输入范围；R29 只是把注入电流限制在约 mA 级。是否长期安全必须结合 STM32 的钳位二极管规格、产品故障条件和 EMC 要求审查。

2.4 ADC 引脚与参考

硬件使用 STM32 的 PC0，即 `ADC12_IN10`：

- 模拟通道：ADC1_IN10；
- 封装引脚：LQFP-100 的 pin 15；
- 软件网络名：`ADC10`；
- ADC 数据：12 位、右对齐；
- VREF+ 与 VDDA 通过 R100 0 Ω 相连；
- VREF−/VSSA 通过 R96 0 Ω 接地；
- ADC 满量程参考约为 +3.3 V，而不是独立精密基准。

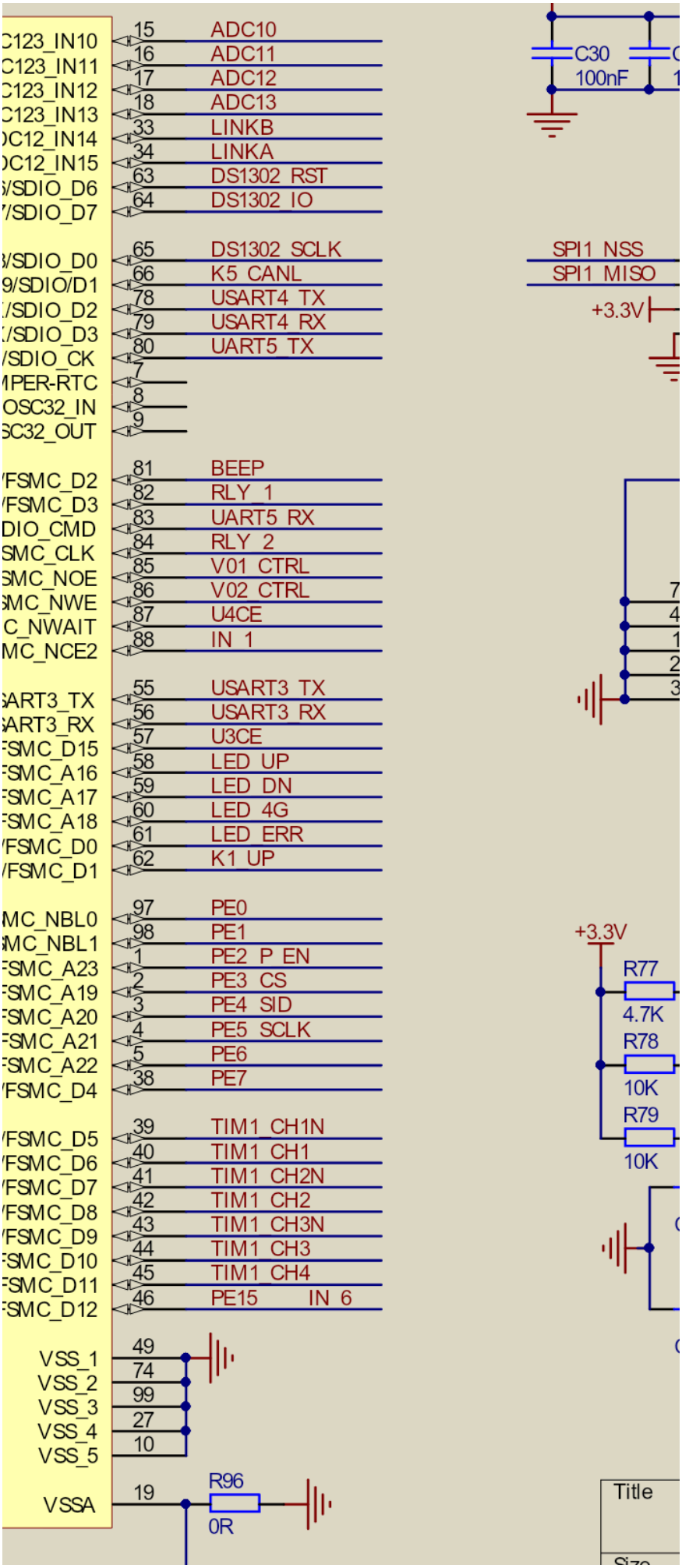

C123_IN10
C123_IN11
C123_IN12
C123_IN13
C12_IN14
C12_IN15
/SDIO_D6
/SDIO_D7
/SDIO_D0
9/SDIO/D1
/SDIO_D2
/SDIO_D3
/SDIO_CK
PER-RTC
OSC32_IN
C32_OUT
FSMC_D2
FSMC_D3
DIO_CMD
SMC_CLK
SMC_NOE
SMC_NWE
C_NWAIT
MC_NCE2
ART3_TX
ART3_RX
FSMC_D15
FSMC_A16
FSMC_A17
FSMC_A18
FSMC_D0
FSMC_D1
MC_NBL0
MC_NBL1
FSMC_A23
FSMC_A19
FSMC_A20
FSMC_A21
FSMC_A22
FSMC_D4
FSMC_D5
FSMC_D6
FSMC_D7
FSMC_D8
FSMC_D9
FSMC_D10
FSMC_D11
FSMC_D12
VSS_1
VSS_2
VSS_3
VSS_4
VSS_5
VSSA
15 ADC10
16 ADC11
17 ADC12
18 ADC13
33 LINKB
34 LINKA
63 DS1302_RST
64 DS1302_IO
65 DS1302_SCLK
66 K5_CANL
78 USART4_TX
79 USART4_RX
80 UART5_TX
7
8
9
81 BEEP
82 RLY_1
83 UART5_RX
84 RLY_2
85 V01_CTRL
86 V02_CTRL
87 U4CE
88 IN_1
55 USART3_TX
56 USART3_RX
57 U3CE
58 LED_UP
59 LED_DN
60 LED_4G
61 LED_ERR
62 K1_UP
97 PE0
98 PE1
1 PE2_P_EN
2 PE3_CS
3 PE4_SID
4 PE5_SCLK
5 PE6
38 PE7
39 TIM1_CH1N
40 TIM1_CH1
41 TIM1_CH2N
42 TIM1_CH2
43 TIM1_CH3N
44 TIM1_CH3
45 TIM1_CH4
46 PE15 IN_6
49
74
99
27
10
19
R96
0R
C30
100nF
SPI1_NSS
SPI1_MISO
+3.3V
+3.3V
R77
4.7K
R78
10K
R79
10K
Title



图 5　STM32 右半部的 ADC10–ADC13 与 SPI 引脚。

同一 ADC 序列还扫描：

软件名	STM32 引脚	ADC 通道	硬件来源	用途
ADC10	PC0 / pin 15	CH10	U4A I-V_MO	电机电流，唯一参与电流算法
ADC11	PC1 / pin 16	CH11	U4B I-V_SUN	GPIO 配置为 AIN，但 ADC 规则序列未包含
ADC12	PC2 / pin 17	CH12	U6A V_SUN	太阳能电压
ADC13	PC3 / pin 18	CH13	U6B +12V’’	电池/输入电压

2.5 理论量程与耗散

分流电压与 ADC 输出：

电机电流	R43 压降	U4A 输出	ADC 码值增量	R43 功耗
0.1 A	5 mV	10 mV	12.4	0.0005 W
0.5 A	25 mV	50 mV	62.1	0.0125 W
1 A	50 mV	100 mV	124.1	0.05 W
2 A	100 mV	200 mV	248.2	0.20 W
3.5 A	175 mV	350 mV	434.4	0.6125 W
5 A	250 mV	500 mV	620.6	1.25 W
10 A	500 mV	1.00 V	1241.1	5 W
20 A	1.00 V	2.00 V	2482.3	20 W
33 A	1.65 V	3.30 V	≈4095	54.45 W

33 A 是“ADC 在 3.3 V 满量程上的理论数学上限”，不是允许通过的机械/热设计电流。实际系统受继电器、F3 PTC、线束和 R43 热能力限制。

在默认 3.5 A 阈值时，R43 已耗散约 0.61 W。若 R2512 的实际额定功率为 1 W，还要考虑环境温度、焊盘铜面积和长期漂移，不能只看瞬时额定。

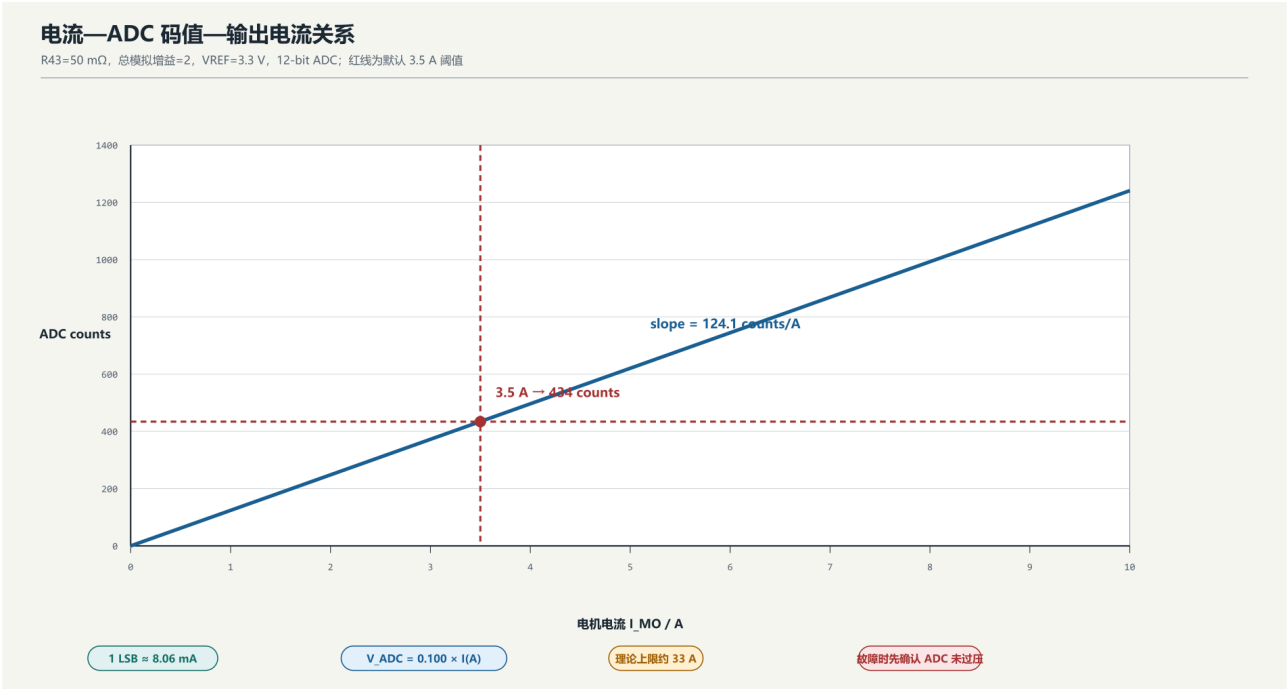


图 6　电流—ADC 码值关系及默认阈值点。

3. ADC 与 DMA 配置

3.1 初始化顺序

`USER/main.c` 中相关顺序为：

1. `dcmotor_init()`：建立继电器 GPIO，默认关闭 RLY_1/RLY_2；
2. `ADC1_Init()`：GPIOC PC0–PC3 模拟输入，配置 ADC1/DMA，并执行 ADC 自校准；
3. `TIM2_Int_Init(7199, 50)`：启动 5 ms 节拍；
4. 主循环：`AD_Process()` 在前，`Dcmotor_Process()` 在后。

DMA 与 ADC 在 `ADC1_Init()` 内即开始连续转换，因此零点校准时硬件已经在持续写 `AD_Value`。

3.2 DMA 缓冲布局

`N=256`、`M=3`，因此 `AD_Value[256][3]` 保存三轮通道循环：

```
AD_Value[n][0] = ADC_Channel_10;  // 电机电流
AD_Value[n][1] = ADC_Channel_12;  // V_SUN
AD_Value[n][2] = ADC_Channel_13;  // +12V''
```

DMA1 Channel1 设置为：

- 外设地址固定：ADC1->DR；
- 内存地址递增；
- 数据宽度 16 bit；
- Circular 循环模式；
- 高优先级；
- 总长度 `N*M=768` 个半字。

3.3 时间预算

SYSCLK 为 72 MHz，ADC 分频 6，ADC 时钟 12 MHz。每通道采样时间 239.5 周期，加转换 12.5 周期，共 252 周期：

$$t_{\text{channel}} = \frac{252}{12\ \text{MHz}} = 21\ \mu s$$

三个通道一轮约 63 μs。256 行 DMA 缓冲约覆盖：

$$256 \times 63\ \mu s \approx 16.13\ \text{ms}$$

主循环每 20 ms 处理一次，因此一次 `raw_avg` 覆盖约一个 20 ms 周期内的最近 16 ms 数据。

3.4 定时器

TIM2 工作在 72 MHz，`PSC=50`、`ARR=7199`：

$$T = \frac{(50+1)(7199+1)}{72\ \text{MHz}} = 5\ \text{ms}$$

`TIM2_IRQHandler()` 每 5 ms 令 `sys_ms += 5`；计数器 `cnt%4==0` 时设置 `flag20ms`。因此 ADC 软件处理周期为 20 ms。

注意：`flag20ms` 是布尔标志而不是事件计数。如果主循环被阻塞超过 20 ms，多个硬件节拍可能只保留一个待处理标志，所以“10 次连续截图”不必然等于严格的 200 ms 墙钟时间。

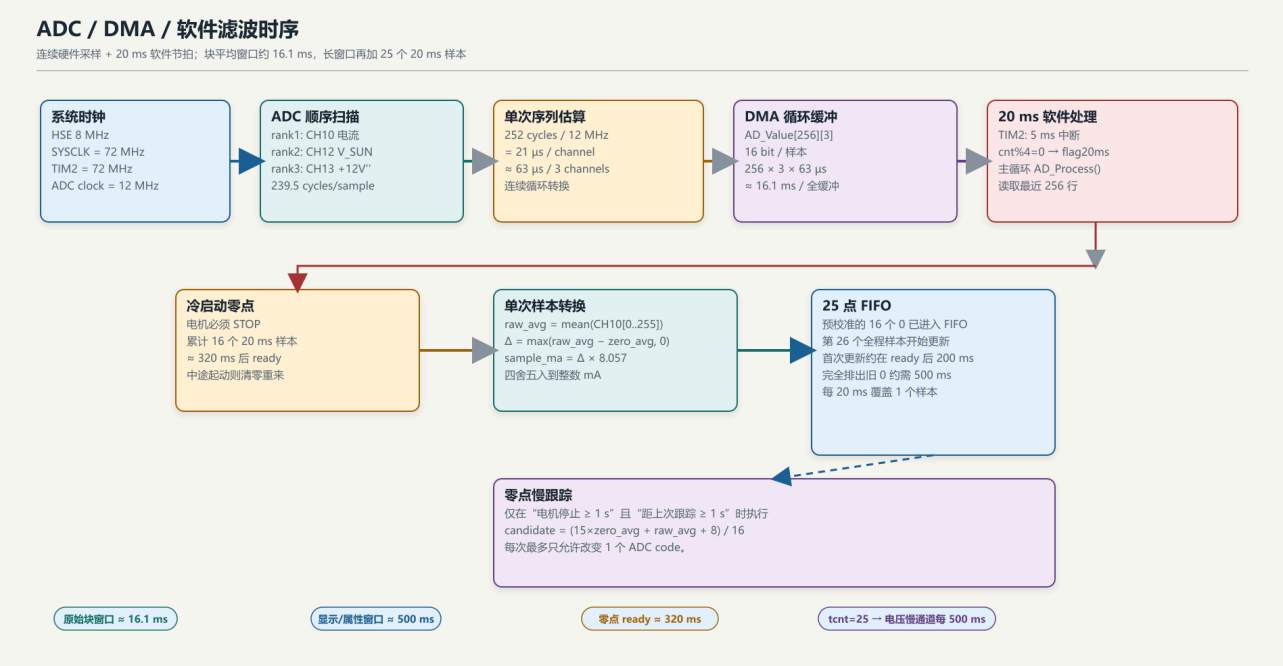


图 7　ADC、DMA 与软件滤波的时序预算。

4. 电流算法

4.1 代码转换常数

`HARDWARE/ADC/adc.h` 定义：

```
#define GAIN          2.0f
#define SAMPLING_RES  0.05f

#define ADC2CURT (float)(3.3f / 4.096f / (SAMPLING_RES * GAIN))
```

`3.3/4.096` 的单位是 mV/count，除以模拟增益后的电流转阻：

$$\text{ADC2CURT} = \frac{3.3}{4.096}\frac{\text{mV}}{\text{count}} \div (0.05\ \Omega \times 2) \approx 8.0566\ \frac{\text{mA}}{\text{count}}$$

即：

$$I_{\text{MA}} = (raw_avg - zero_avg) \times 8.0566$$

该常数与当前 R43、R23/R22、R27/R25 网络一致。

4.2 原始块平均

每个 20 ms 处理点执行：

```
for (count = 0; count < N; count++)
    sum += AD_Value[count][0];
raw_avg = sum / N;
```

256 点求和无溢出风险：最大 `256×4095=1,048,320`，可放入 `u32`。该平均减少白噪声和继电器抖动，但 DMA 仍在运行，因此读取过程中缓冲最早与最新行可能不是同一绝对时刻的快照。

4.3 冷启动零点

代码要求以下三个条件同时成立才建立零点：

- `s_current_zero_ready == 0`；
- `sys_moter_state == MOTOR_STOP`；
- 已经收到 16 个 20 ms 块样本。

完成后：

$$zero_avg = \text{round}\left(\frac{1}{16}\sum_{k=1}^{16} raw_avg[k]\right)$$

名义耗时约 320 ms。`Dcmotor_Process()` 在 ready 之前会强制状态为 STOP 并关断 RLY_1，因此正常路径下不会在未完成零点时运行电机。

4.4 单次电流

ready 后每个 20 ms：

```
if (raw_avg >= zero_avg)
    sample_ma = (u16)((float)(raw_avg - zero_avg) * ADC2CURT + 0.5f);
else
    sample_ma = 0;
```

低于零点被钳位为 0，所以算法只表达单方向电流；电机反电动势或继电器切换引起的负向瞬态不会形成负电流读数。

`sample_ma` 是“一个 20 ms 调度点的约 16 ms 块平均”，不是单个 ADC 瞬时值。

4.5 25 点滑动平均

`ADC_CURRENT_AVG_WINDOW=25` 的 FIFO 保存 25 个 `sample_ma`：

- 每 20 ms 更新一次；
- 25 个样本覆盖约 500 ms；
- `armReg.motor_curr` 与 `g_adc_val[0]` 使用该平均；
- 前 25 个样本只填充 FIFO；
- 第 26 个样本才开始把平均结果写回 `armReg.motor_curr`。

需要注意，FIFO 更新代码位于零点 ready 分支之外：冷启动校准阶段的 16 个 `sample_ma=0` 已经进入 FIFO。因此：

- 第 26 个全局处理样本（约 ready 后 200 ms）开始写回平均电流；
- 此时窗口里仍有 15 个校准期 0，首次读数会明显偏低；
- 再经过约 500 ms，窗口才完全由 ready 后的真实样本组成。

电机若在 ready 后立即启动，前约 200 ms 的属性值仍可能是 0，随后从偏低值逐步爬升到稳定值；保护逻辑使用的是 `sample_ma`，不受这段显示爬升影响。

因此 `armReg.motor_curr` 的显示/上报窗口相对保护用的短期样本明显更慢，这是有意分离的两条路径。

4.6 零点慢跟踪

当电机停止累计 ≥1000 ms，并且距离上次跟踪 ≥1000 ms 时：

```
candidate = (15 * zero_avg + raw_avg + 8) / 16;
```

随后每次只允许变化 1 个 ADC code。该策略可以缓慢补偿温漂和电源漂移，同时避免运行中把负载电流误当成新零点。

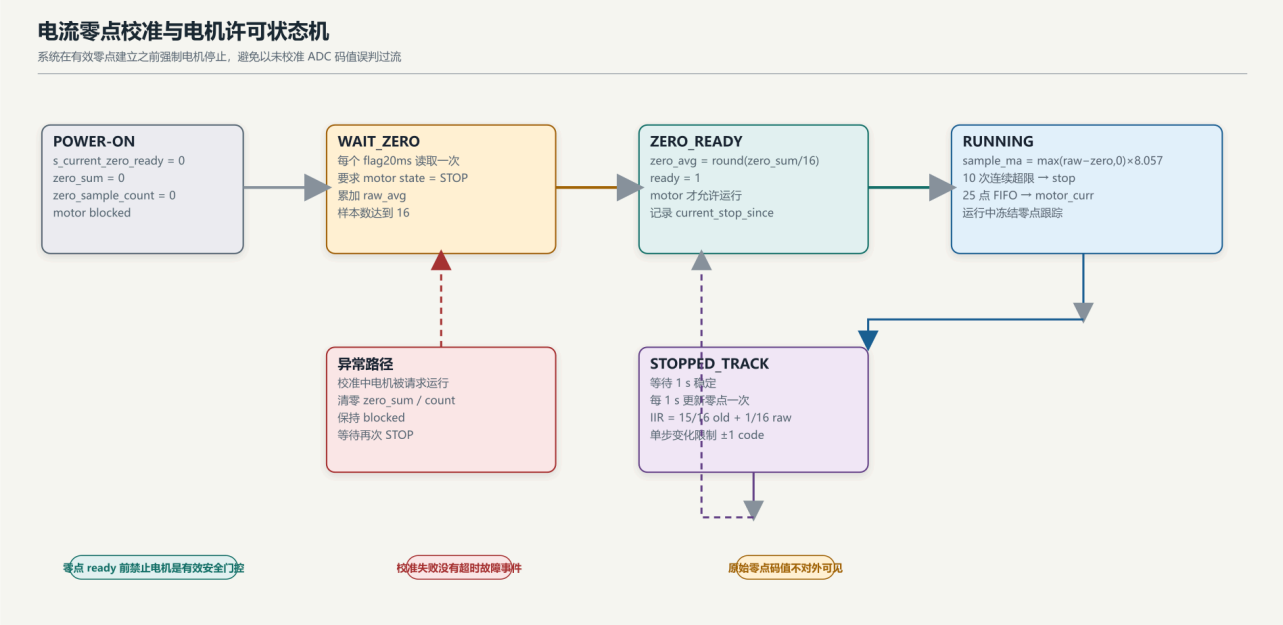


图 8　零点校准、运行许可和慢跟踪状态。

5. 堵转与过流保护

5.1 判定路径

保护没有使用 500 ms FIFO 平均，而是使用短期 `sample_ma`：

```
if (sample_ma > armReg.motor_warning_curr_upper) {
    stall_detect_count++;
    if (stall_detect_count >= 10) {
        EventPostMotorStall(sample_ma);
        dcmotor_request_stop(MOTOR_STOP_REASON_STALL);
        stall_detect_count = 0;
    }
} else {
    stall_detect_count = 0;
}
```

保护动作顺序：

1. 第 N 次超限时递增计数器；
2. 任意正常样本使计数器清零，要求连续超限；
3. 第 10 次触发 `motor_stall` 事件；
4. `dcmotor_request_stop()` 设置 `pending_stop_reason=STALL`、将状态切到 STOP，立即关闭 RLY_1；
5. 下一次 `Dcmotor_Process()` 因状态变化发送 `motor_stop`，reason 为 `stall`，同时排队属性上报。

即使 4G 离线导致 `EventPostMotorStall()` 返回 0，停机动作仍会执行，安全动作不依赖网络。

5.2 时间估算

- 单个处理周期：20 ms；
- 第 1 次超限到第 10 次超限：9 个间隔，约 180 ms；
- 按 10 个完整的 20 ms 统计窗口计：约 200 ms；
- 加上主循环可能拥塞、事件排队和继电器机械释放时间，实际停机延迟会更大。

因此，代码注释中的“约 300 ms”不准确；更合适的描述是“10 个处理点，名义约 180–200 ms 的软件判定期”。真正安全关断时间应在样机上从电机电流阶跃开始，到线圈断电、触点断开和电机电流下降到零逐一测量。



图 9　连续 10 次超限的堵转判定时序。

5.3 停机和保护属性

`dcmotor_stop()` 仅关闭 RLY_1，不清除 RLY_2 方向状态。下一次上升/下降会先重设方向，再开启 RLY_1，逻辑上可接受。

`MOTOR_STOP_REASON_STALL` 为枚举值，`EventPostMotorStop()` 会转换成字符串 `stall`。客户端会看到：

- `motor_stall` 事件，参数包含 `motor_current`，单位 mA；
- `motor_stop` 事件，参数包含 `reason=stall` 与当前开度。

如果没有独立硬件过流锁存，停机后只要固件再次请求运行且当前条件解除，就有可能重新启动。产品层需明确堵转后是否允许自动重试。

6. 固件结构与代码位置

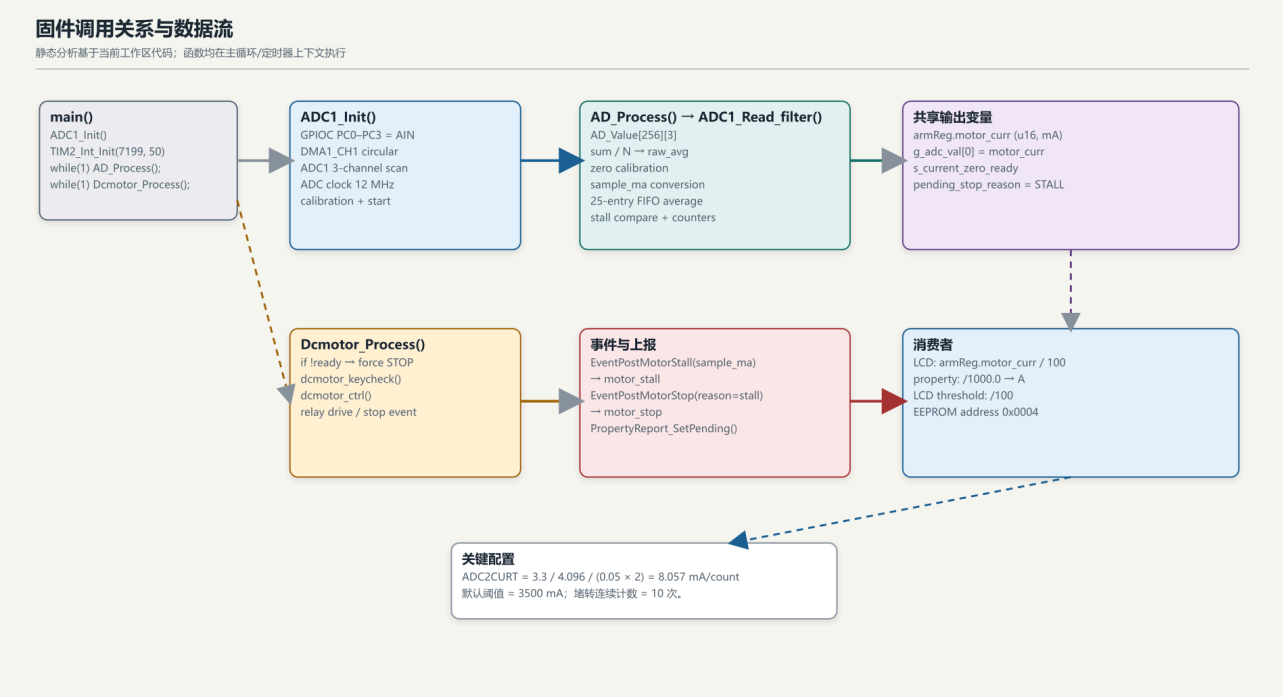


图 10　电流采样、停机与上报的调用关系。

功能	文件与行号	说明
ADC 常数	`HARDWARE/ADC/adc.h:15-20`	`ADC2CURT`、`GAIN=2.0`、`SAMPLING_RES=0.05`
ADC GPIO	`HARDWARE/ADC/adc.c:36-43`	PC0–PC3 模拟输入
ADC/DMA	`HARDWARE/ADC/adc.c:47-111`	CH10/12/13、DMA 循环、12 MHz ADC
主处理入口	`HARDWARE/ADC/adc.c:149-154`	`AD_Process()` 调用新滤波器
块平均/零点	`HARDWARE/ADC/adc.c:283-348`	256 样本平均、16 次零点、慢跟踪
转换与 FIFO	`HARDWARE/ADC/adc.c:321-378`	mA 换算、25 点平均
堵转判定	`HARDWARE/ADC/adc.c:382-403`	10 次连续超限
ADC 就绪门控	`HARDWARE/DC_MOTOR/dc_motor.c:101-123`	零点未完成时强制停止
继电器方向	`HARDWARE/DC_MOTOR/dc_motor.c:54-91`	K2/K1 顺序控制
5 ms 节拍	`HARDWARE/TIMER/timer.c:213-245`	20 ms 标志每 4 个 5 ms tick
初始化顺序	`USER/main.c:40-54`	电机 GPIO → ADC → TIM2
主循环顺序	`USER/main.c:76-80`	先采集，后电机处理
默认阈值	`HARDWARE/REG/reg.c:229-236`	3500 mA
阈值上电迁移	`HARDWARE/REG/reg.c:103-125`	`<1000` 自动乘以 100
4G 阈值配置	`HARDWARE/GPRS/gprs_JSON.c:194-204`	`MotorMaxCurrConfig`
阈值 LCD 设置	`HARDWARE/Lcd12864/Dispaly.c:539-550`	输入值 ×100 后保存
堵转事件	`HARDWARE/GPRS/event_pub.c:74-84`	`motor_stall` 带 mA
停机事件	`HARDWARE/GPRS/event_pub.c:87-119`	停机原因文字

功能	文件与行号	说明
属性上报	`HARDWARE/GPRS/property_report.c:29-42`	mA / 1000 → A
LCD 显示	`HARDWARE/Lcd12864/Dispaly.c:461`	mA / 100 → 3.5 A 形式

7. 显示、属性与事件的数据一致性

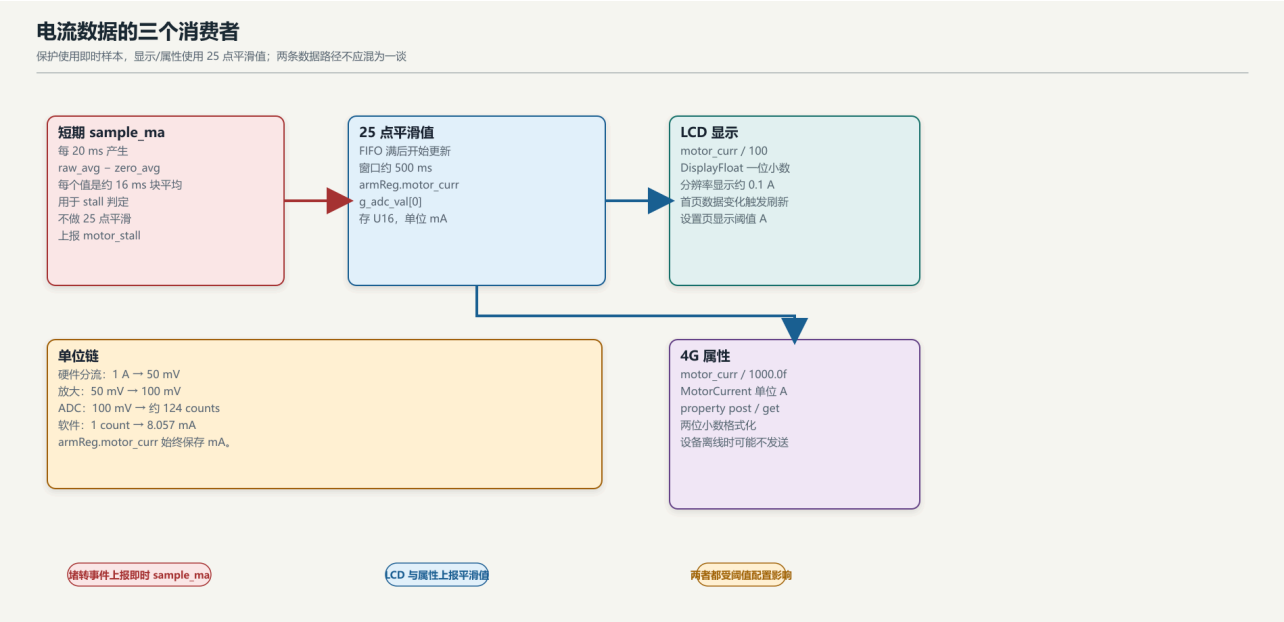


图 11　即时保护路径与平滑显示/上报路径。

7.1 LCD

首页：

```
DisplayFloat(&motor[5], armReg.motor_curr / 100, 0);
```

`3500 / 100 = 35`，`DisplayFloat` 再格式化为 `3.5`，因此显示分辨率约 0.1 A。该值来自 25 点滑动平均。

设置页：

```
DisplayFloat(&duzhuan[6], armReg.motor_warning_curr_upper / 100u, 1);
```

阈值显示同为 A，一位小数。

7.2 4G 属性

```
para->MotorCurrent = armReg.motor_curr / 1000.0f;
```

输出字段 `MotorCurrent` 的单位为 A，格式化两位小数。由于硬件/软件分辨率为约 8 mA，属性值可保留到 0.01 A，但低电流下仍受 ADC 噪声限制。

7.3 堵转事件

```
snprintf(para_buf, sizeof(para_buf),
         "\"motor_current\":%u", motor_current_ma);
return replyEven("motor_stall", para_buf, "");
```

事件携带的是触发时的 `sample_ma`，不是 25 点平滑值。这样更接近保护判定数据，但会与同时上报的 `MotorCurrent` 属性存在差异，属于设计上需要明确的语义。

8. 阈值配置与 EEPROM

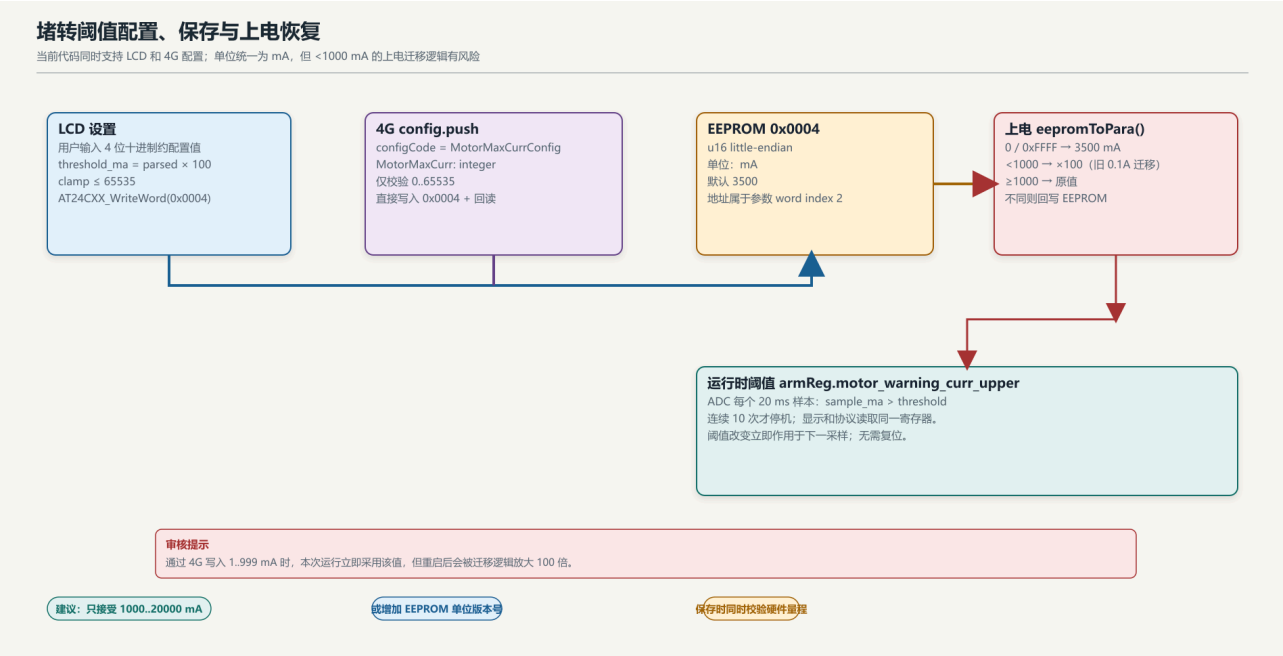


图 12　阈值从 LCD/4G 保存、上电迁移到运行时的路径。

8.1 默认值与存储位置

- 结构体字段：`armReg.motor_warning_curr_upper`，`u16`，单位 mA；
- 默认值：3500 mA；
- EEPROM 字节地址：`0x0004`；
- 该地址对应参数字序号 2；
- 上电读取：`eepromToPara()`；
- 修改后无需重启即可生效；
- 通过 `Config_WriteWordWithVerify()` 写入并回读校验。

8.2 两条配置入口

LCD

用户输入被解析为十进制值并乘以 100，再限制到 16 位并写入 EEPROM。显示和保存在 A 分辨率上。

4G `config.push`

JSON 示例：

```
{
  "configCode": "MotorMaxCurrConfig",
  "params": {
```

```
    "MotorMaxCurr": 3500
  }
}
```

`Config_ParseU16()` 只检查：

- 数字类型；
- `0..65535`；
- 没有小数。

它不检查机械/热允许范围，也不检查最小可分辨电流。因此 `0`、`1`、`999`、`65535` 都能被接受并写入。

8.3 重要缺陷：小于 1000 mA 的重启迁移

`eepromToPara()` 中：

```
if (stored_current_limit == 0u || stored_current_limit == 0xFFFFu)
    armReg.motor_warning_curr_upper = 3500u;
else if (stored_current_limit < 1000u)
{
    /* 兼容旧0.1A编码（如35表示3.5A） */
    migrated = (u32)stored_current_limit * 100u;
    ...
}
```

假设通过 4G 成功写入 `500`（即 0.5 A）：

- 本次运行：阈值立即为 500 mA；
- 重启后：500 被视为旧编码 `50.0 A`，迁移为 50,000 mA；
- 结果：实际过流保护从 0.5 A 被意外放宽到 50 A。

这是一个优先级很高的正确性缺陷。兼容旧单位必须使用版本号或明确的 EEPROM 标记，不能只根据数值大小猜测。

9. 代码审查发现

P0：安全或功能正确性

P0-1　低于 1000 mA 的配置重启后放大 100 倍

证据：`reg.c:108-125` 与 `gprs_JSON.c:194-204`。

建议：

- 统一只接受合理 mA 范围（例如 1000–20000 mA）；
- 或在 EEPROM 中加入单位版本号，彻底删除“按数值猜测单位”；
- 对 LCD/4G 两条写入路径使用同一个配置校验函数。

P0-2　软件保护不能替代短路保护

名义判定需要 10 个处理点，约 180–200 ms，再加继电器机械延迟。继电器、PTC、线束与分流电阻不能承受持续短路。

建议：

- 在硬件上增加快速限流/熔断/电子开关；
- 评估把保护决策移入 DMA 半完成/定时器中断，并记录真实 `sys_ms`；
- 对短路、堵转和启动浪涌分别定义阈值/时延。

P0-3　ADC 输入过压

LM358 由 +5 V 供电，U4A 输出可以在极高故障电流下超过 STM32 的 3.3 V 模拟输入绝对最大值。R29 1 kΩ 仅提供一定限流。

建议：

- ADC 前增加低漏电肖特基钳位到 3.3 V，并核算注入电流；
- 或将 U4A 改为 3.3 V 单电源并重新核算共模范围；
- 加故障电阻/保险，避免分流、运放或继电器故障导致 ADC 过压。

P1：测量可靠性与诊断

1. **零点无失败状态**：校准只返回 ready/not ready。若样本长期不满足条件，电机永久被阻止，但没有 `ADC calibration timeout` 事件。
2. **无饱和/开路检测**：ADC 满码、opamp 饱和、R43 开路、U4A 失效、PC0 脱焊都缺少独立故障状态。
3. **负向瞬态被钳零**：`raw_avg < zero_avg` 时直接置 0，电机反电动势和关机回摆不会形成负读数。
4. **保护使用的是处理次数而不是墙钟窗口**：主循环拥塞时可超过名义 200 ms。
5. **无首个启动阶段抑制**：如果继电器吸合浪涌持续超过约 200 ms，可能被当堵转；反之短时尖峰可被滤除，这需要产品时延要求决定。

6. **显示/属性启动慢**：零点校准阶段的 16 个零样本预先占用 FIFO，电机 ready 后立即运行时，平滑电流要经过约 200 ms 才首次更新，并在约 500 ms 后稳定；早期属性值可能为 0 或偏低。

P2：可维护性与代码质量

1. `flag20ms` 在 TIM2 中断写、主循环读，但没有 `volatile`，跨编译单元优化下不应依赖隐式可见性。
2. 多处注释与代码不一致：
   - `TIM2_Int_Init` 注释写 10 ms，实际 5 ms；
   - `ADC_NbrOfChannel` 注释写 4，实际 3；
   - `N` 注释写每通道 128 次，实际 256；
   - 堵转注释写约 300 ms，实际名义 180–200 ms；
   - `adc.h` 旧公式注释写“除以 0.12”，宏实际使用 0.1。
3. `ADC1_Read_filter_old()`、`calc_adc_val()`、`ADC_ConvertedValue`、`REFER[0]=1580` 属于遗留路径或未使用状态，建议删除或在注释中明确保留原因。
4. 保护事件只带 `motor_current`，不带 `threshold`、`raw_avg`、`zero_avg`，现场定位信息有限。
5. `EventPostMotorStall()` 只有在 4G ready 且 `autoFlag==0` 时才入队；安全停机不受影响，但事件可能丢失。

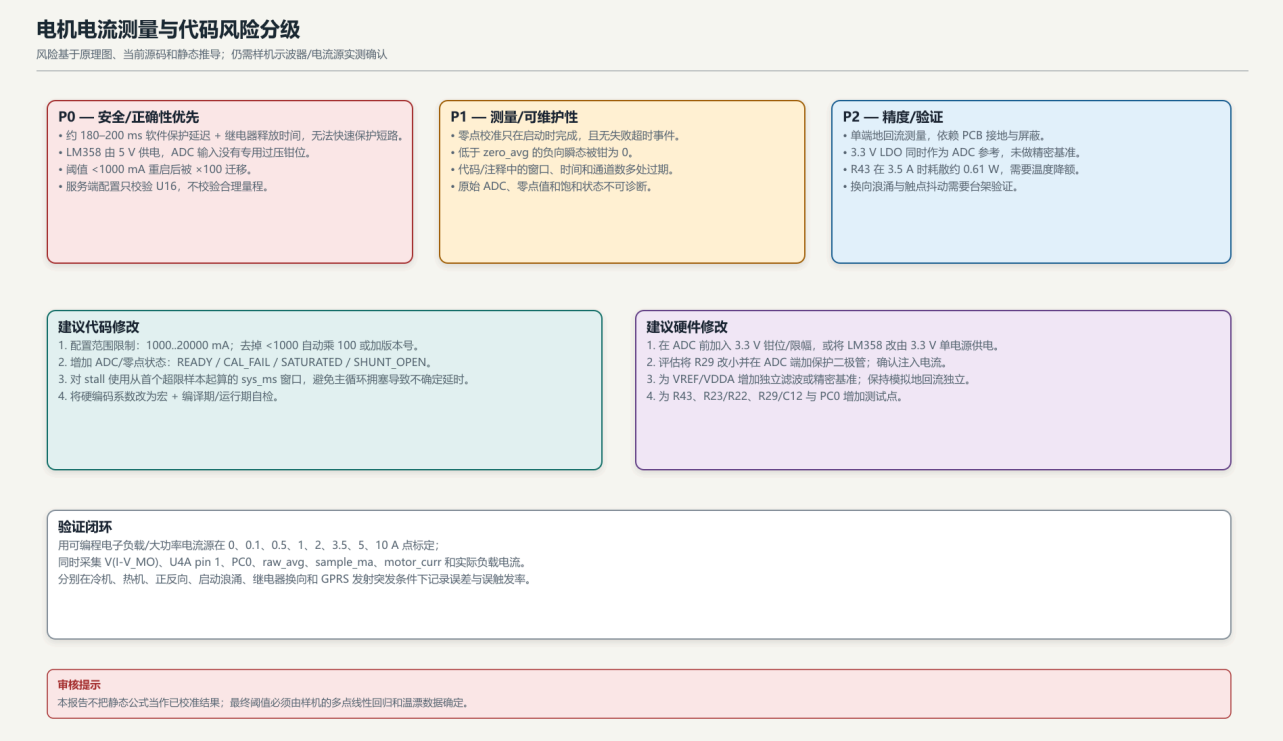


图 13　风险分级与改进优先级。

10. 标定与测试方案

10.1 硬件静态检查

1. 断电测 R43 两端到 GND、I-V_MO 到 U4A pin 3、U4A pin 1 到 PC0 的通断。
2. 检查 R23/R22、R25/R27 的实际值和焊装状态；R22/R25 不能错装。
3. 在空白板或维修板上验证 R43 的 Kelvin 取样是否从焊盘内侧引出，避免把走线铜阻和焊锡阻计入。
4. 检查 C12 是否靠近 ADC 引脚，R29 是否在运放与 C12 之间。

10.2 零点与噪声

- 电机完全停止，RLY_1/RLY_2 均断开，记录 PC0 原始码值 10 分钟；
- 统计均值、峰峰值、标准差和温漂；
- 比较 `ADC_CURRENT_DEBUG=1` 下的 `raw / zero / sample / smooth / ready`；
- 在 4G 发射、LCD 刷新、EEPROM 写入时重复，检查数字噪声耦合；
- 确认零点 ready 时间约 320 ms，而不是无限等待。

10.3 多点电流标定

用可编程电子负载或大功率低噪声电流源，在以下点记录：

设定电流	期望 V_ADC	期望码值	实测码值	误差
0 A	0 mV	0		
0.1 A	10 mV	12.4		
0.5 A	50 mV	62.1		
1 A	100 mV	124.1		
2 A	200 mV	248.2		
3.5 A	350 mV	434.4		
5 A	500 mV	620.6		
10 A	1 V	1241.1		

建议至少正、反两个方向各测一次，以确认 K1 换向没有引入额外零偏或地电位变化。

10.4 阶跃和堵转测试

1. 从一个已知小电流阶跃到 3.5 A 以上，记录：
    - 实际分流电压；
    - U4A pin 1；
    - ADC raw 值；
    - `sample_ma`；
    - RLY_1 驱动；
    - 负载电流消失时刻。

2. 重复 100 次，记录误触发、漏触发和最大停机时间。
3. 在启动浪涌、继电器换向、GPRS 发射和 LCD 刷新并发时重复。
4. 对 0.5、1、2、3、4、5 s 的短时过载分别验证，确认“堵转”与“正常启动”的区分。

10.5 阈值持久化回归

至少测试以下配置并重启：

- `0`：当前运行行为与重启后默认值是否一致；
- `1`、`500`、`999` mA：验证当前已知迁移缺陷；
- `1000`、`3500`、`10000` mA：验证正常保存与回读；
- `65535` mA：验证上限和 UI/JSON；
- 通过 LCD 与 4G 分别设置同一值，确认两条入口一致。

10.6 故障注入

- 断开 R43 或模拟分流开路；
- 将 I-V_MO 拉到 0 V、3.3 V、5 V；
- 断开 PC0；
- 让 U4A 输出饱和；
- 在电机运行中模拟 4G 离线；
- 让 25 点 FIFO 长时间全为 0，再突然恢复到 3 A；
- 检查是否会误堵转、漏堵转，以及 ADC 是否超过绝对最大额定值。

11. 建议的代码级改进

11.1 单位与配置

建议把阈值统一为明确的工程单位，并增加范围校验：

```
#define MOTOR_CURRENT_MIN_MA  1000u
#define MOTOR_CURRENT_MAX_MA  20000u

static bool motor_current_limit_valid(u16 ma)
{
    return ma >= MOTOR_CURRENT_MIN_MA &&
           ma <= MOTOR_CURRENT_MAX_MA;
}
```

并移除仅凭 `<1000` 判断旧单位的逻辑，改用 EEPROM 版本或专用标志。

11.2 真实时间窗口

保护计数可以增加起始时间戳：

```
if (sample_ma > threshold) {
    if (stall_start_ms == 0)
        stall_start_ms = sys_ms;
    if ((u32)(sys_ms - stall_start_ms) >= MOTOR_STALL_CONFIRM_MS)
        motor_stop_for_stall();
} else {
    stall_start_ms = 0;
}
```

这样软件时延与主循环调度次数解耦。阈值和确认时间应作为可配置参数并参考硬件能力。

11.3 状态接口

建议暴露至少以下状态：

```
typedef enum {
    ADC_CUR_INIT = 0,
    ADC_CUR_READY,
    ADC_CUR_ZERO_FAIL,
    ADC_CUR_SATURATED,
```

```
    ADC_CUR_OPEN_SENSOR
} AdcCurrentState;
```

当电机已指令运行而电流长期低于“运行最小电流”时，可以单独判定负载断线/分流断线，而不是继续把它当成 0 A。

11.4 诊断事件

`motor_stall` 建议增加：

- `motor_current`：触发样本；
- `threshold`：当前阈值；
- `raw_avg`、`zero_avg`；
- `confirm_ms`；
- 固件版本和采样通道。

这不会改变停机优先级，但能显著降低现场误判成本。

11.5 编译期自检

```
_Static_assert(N == 256, "DMA buffer size changed");
_Static_assert(M == 3, "ADC scan count changed");
```

并把 `ADC2CURT` 的计算拆成命名常数，防止 R43、R22/R23、R25/R27 中任一变化而软件未同步。

12. 关键结论

1. 当前硬件与 `GAIN=2.0` 的软件假设一致，理论换算约为 `8.06 mA/ADC code`。
2. `sample_ma` 用于保护，`armReg.motor_curr` 用于显示与属性，这是两个不同时间尺度的数据。
3. 启动前必须完成零点校准；这是有效的安全门控。
4. 保护判定是 10 个连续处理点，通常约 180–200 ms，而不是一次 ADC 立即停机。
5. 当前最需要修复的不是基本换算，而是阈值持久化、ADC 过压保护和诊断/延迟可观测性。
6. 任何用于生产参数的增益、阈值、温漂和响应时间，都必须通过实物多点标定确认。

附录 A　关键源码摘录

A.1 ADC 换算常数

```
#define ADC2CURT (float)(3.3f / 4.096f / (SAMPLING_RES * GAIN))
#define GAIN 2.0f
#define SAMPLING_RES 0.05f
```

A.2 零点与单次样本

```
if (!s_current_zero_ready) {
    if (sys_moter_state == MOTOR_STOP) {
        zero_sum += raw_avg;
        zero_sample_count++;
        if (zero_sample_count >= ADC_CURRENT_ZERO_SAMPLE_COUNT) {
            zero_avg = (zero_sum + 8u) / ADC_CURRENT_ZERO_SAMPLE_COUNT;
            s_current_zero_ready = 1;
        }
    }
}

sample_ma = (raw_avg >= zero_avg)
          ? (u16)((float)(raw_avg - zero_avg) * ADC2CURT + 0.5f)
          : 0;
```

A.3 25 点 FIFO

```
if (cnt < ADC_CURRENT_AVG_WINDOW) {
    val[index] = sample_ma;
    isum += sample_ma;
    cnt++;
} else {
    isum -= val[index];
    val[index] = sample_ma;
    isum += sample_ma;
    armReg.motor_curr = isum / ADC_CURRENT_AVG_WINDOW;
    g_adc_val[0] = armReg.motor_curr;
}
```

A.4 堵转分支

```
if (sample_ma > armReg.motor_warning_curr_upper) {
    stall_detect_count++;
    if (stall_detect_count >= 10) {
        EventPostMotorStall(sample_ma);
        dcmotor_request_stop(MOTOR_STOP_REASON_STALL);
        stall_detect_count = 0;
    }
} else {
    stall_detect_count = 0;
}
```

A.5 停机动作

```
void dcmotor_request_stop(MotorStopReason reason)
{
    pending_stop_reason = reason;
    sys_moter_state = MOTOR_STOP;
    dcmotor_stop();
}
```

附录 B　原始原理图局部与整页

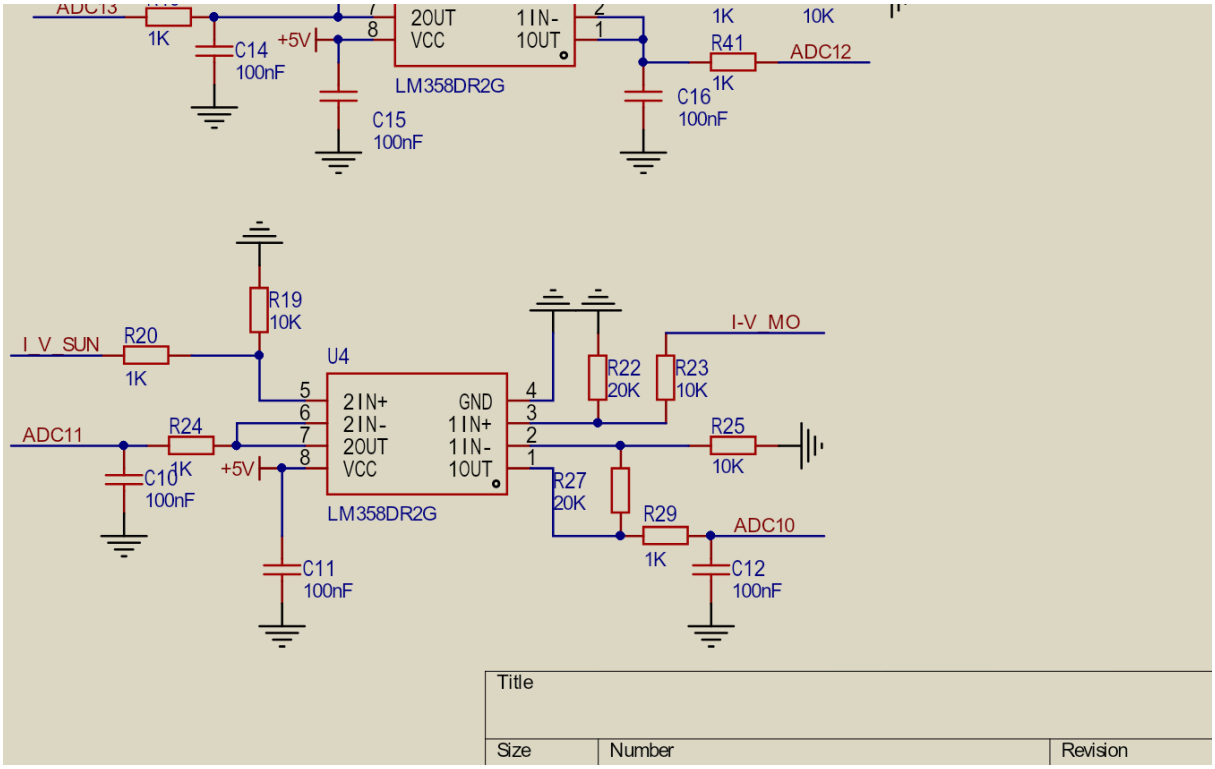


图 B1　U4A 与 ADC10 原始局部。

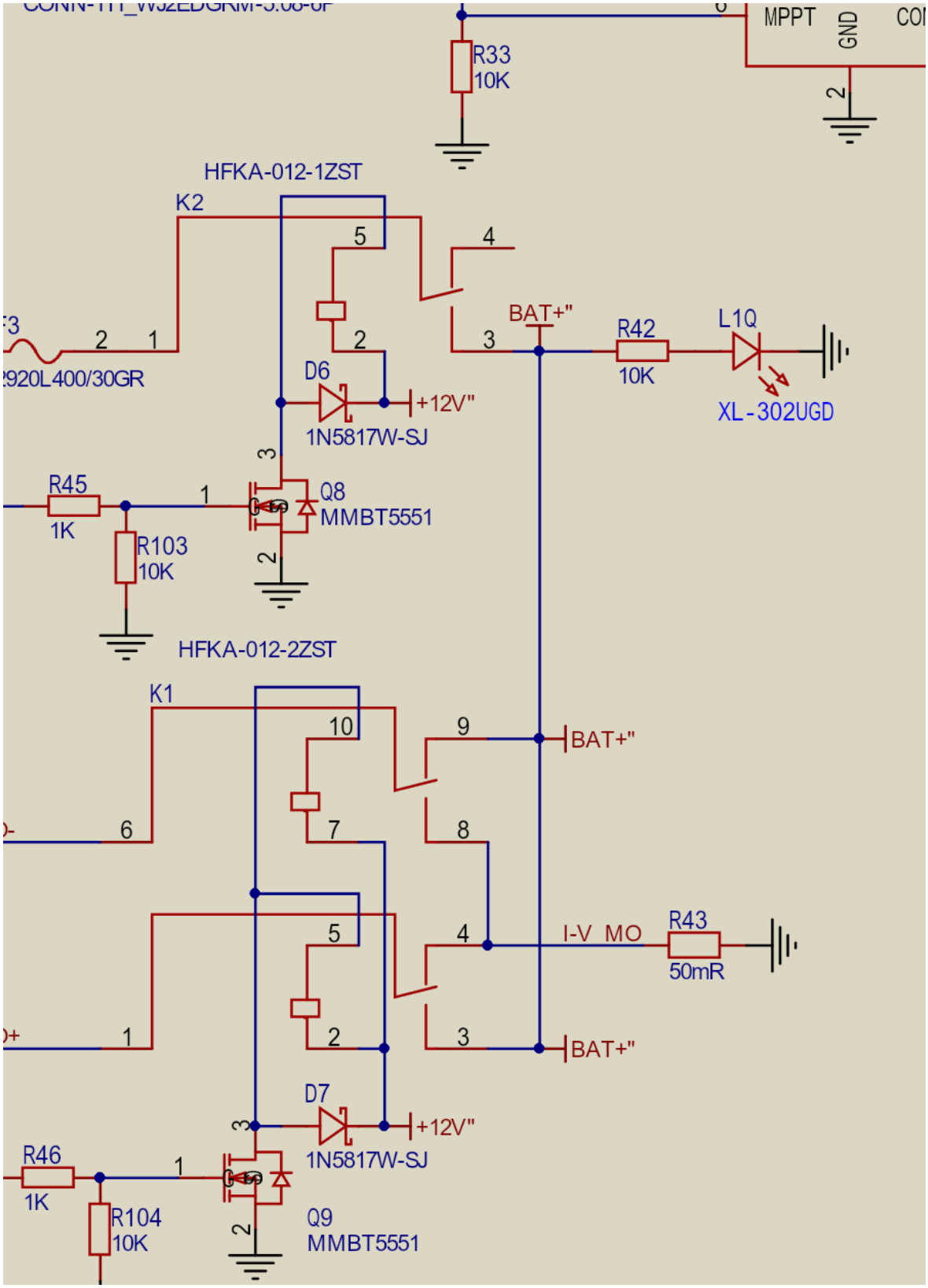


图 B2　K1、MO±、I-V_MO、R43 原始局部。

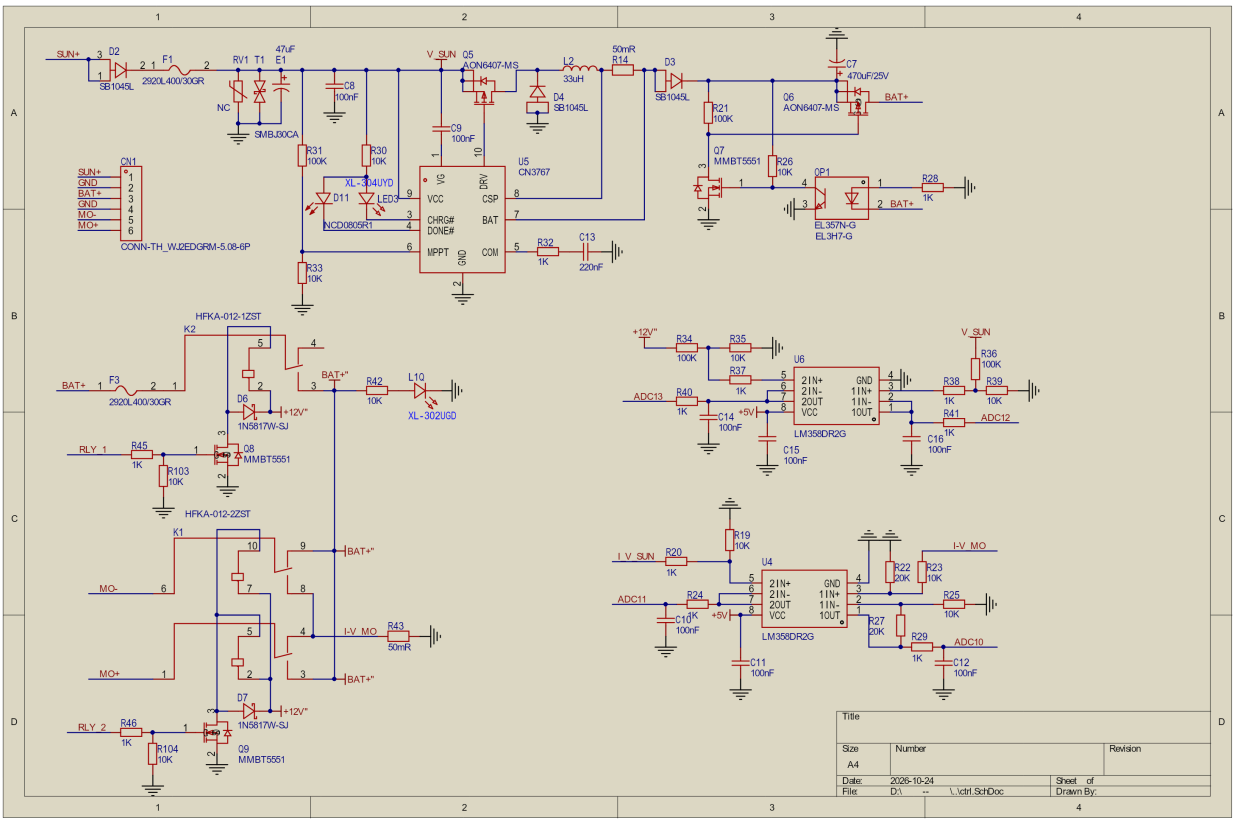


图 B3　第 2 页完整控制/电源/测量原理图。

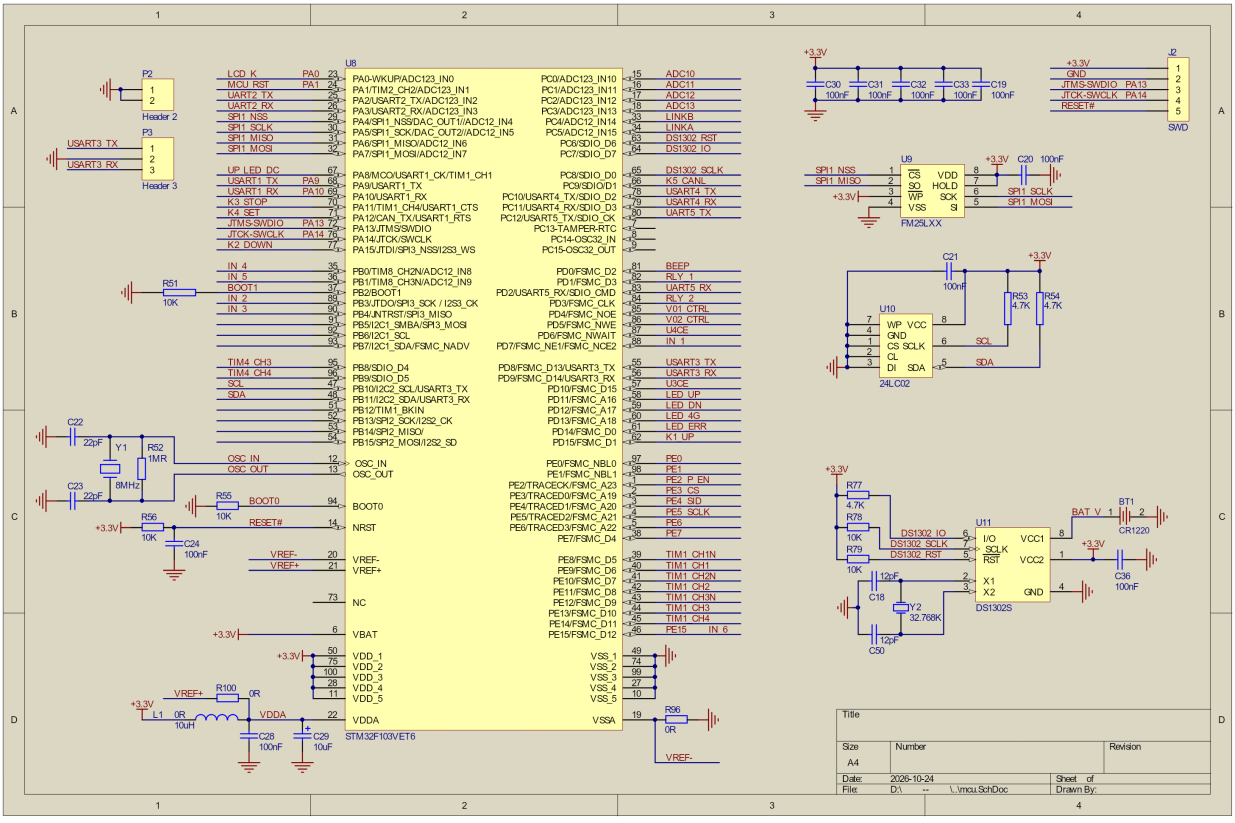


图 B4　第 5 页完整 MCU/ADC 引脚原理图。

附录 C　报告资产

文件	内容
`QIBIJI_RTU_V1.1_电机电流测定电路与固件分析.pdf`	固定版式报告
`QIBIJI_RTU_V1.1_电机电流测定电路与固件分析.docx`	可编辑报告
`QIBIJI_RTU_V1.1_电机电流测定电路与固件分析.md`	Markdown 源文本
`motor_current_report_assets/`	10 张分析图、SVG 源图与原理图局部

> 报告中的“ADC 满码”“理论 33 A”“8.06 mA/LSB”均为理想值。实际标定应使用实际 VREF、电阻容差、LM358 输入失调/摆幅、R43 温漂、PCB 接地和继电器机械时序进行修正。